

Ethereum Bugs through the Lens of Formal Verification

Shelly Grossman, Ittai Abraham, Guy Golan-Gueta,
Y. Michalevsky, Noam Rinetzky, Mooly Sagiv and Yoni Zohar

whoami

- Phd in security from Stanford
- CTO and co-founder @Anjuna (www.anjuna.io)

Smart Contracts

- Smart Contracts are actual computer programs that can be stored on a blockchain
- Executed by miners that form a consensus on the resulting state post execution
- Simple contracts can be implemented on top of blockchains like  **bitcoin** or  **litecoin**
-  **ethereum**: a flexible and popular smart contracts framework over its own blockchain

Powerful but tricky

The \$280M Ethereum's Parity bug.

A critical security vulnerability in Parity multi-sig wallet got triggered on 6th November—paralyzing wallets created after the 20th July.

As you may have read, Parity issued a security advisory today to inform its users and developers about a bug that got “*accidentally*” triggered, which resulted in freezing more than \$280M worth of ETH, including funds belonging to Parity's Founder & Ethereum former core developer, Gavin Wood.

The DAO Attacked: Code Issue Leads to \$60 Million Ether Theft



Michael del Castillo   
Jun 17, 2016 at 14:00 UTC

NEWS

The DAO, the distributed autonomous organization that had collected over \$150m worth of the cryptocurrency ether, has reportedly been hacked, sparking a broad market sell-off.

A [leaderless organization](#) comprised of a series of smart contracts written on the ethereum codebase, The DAO has lost [3.6m ether](#), which is currently sitting in a separate wallet after being split off into a separate grouping dubbed a "child DAO".

The DAO Bug

Object DAO

Map<Object, int> credit

int balance

Invariant (sum o: credit[o]) = balance

Method withdrawAll(Object o)

2: if (oCredit > 0)

// 2.5: credit[o] = 0

3: this.balance -= oCredit

4: o.pay(oCredit)

5: credit[o] = 0

Method deposit(Object o, int amount)

6: credit[o] += amount

7: this.balance += amount

Expected invariant: sum of credits should equal the balance

The DAO Bug

Object GoodClient

Object Dao, int balance

Method init(Object dao)

1: this.Dao = dao

Method pay(int profit)

2: this.balance += profit

Method depositCredit(Object dao, int amount)

3: Dao.deposit(this, amount)

Method getCredit(Object dao)

4: Dao.withdrawAll(this)

Doesn't violate the invariant

Object Attacker

Object Dao, bool stop, int balance

Method init(Object dao)

1: Dao = dao

2: stop = false

Method pay(int profit)

3: this.balance += profit

4: if (!stop)

5: stop = true

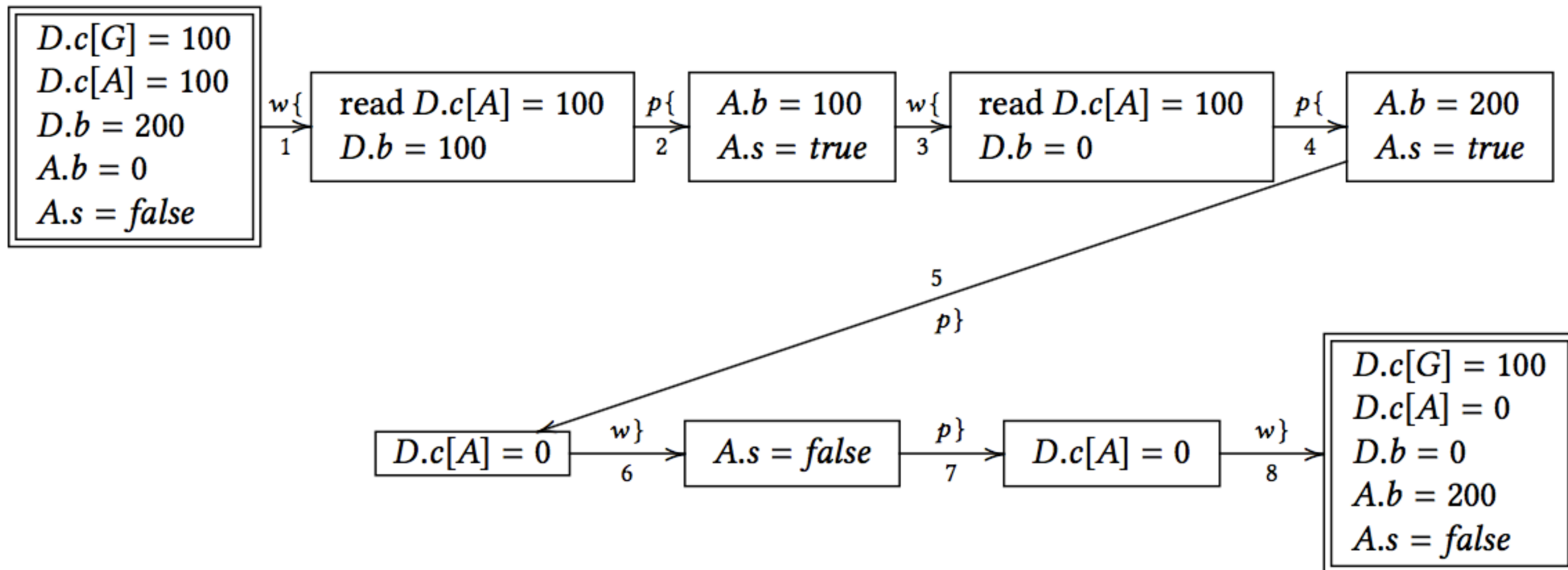
6: Dao.withdrawAll(this)

7: stop = false

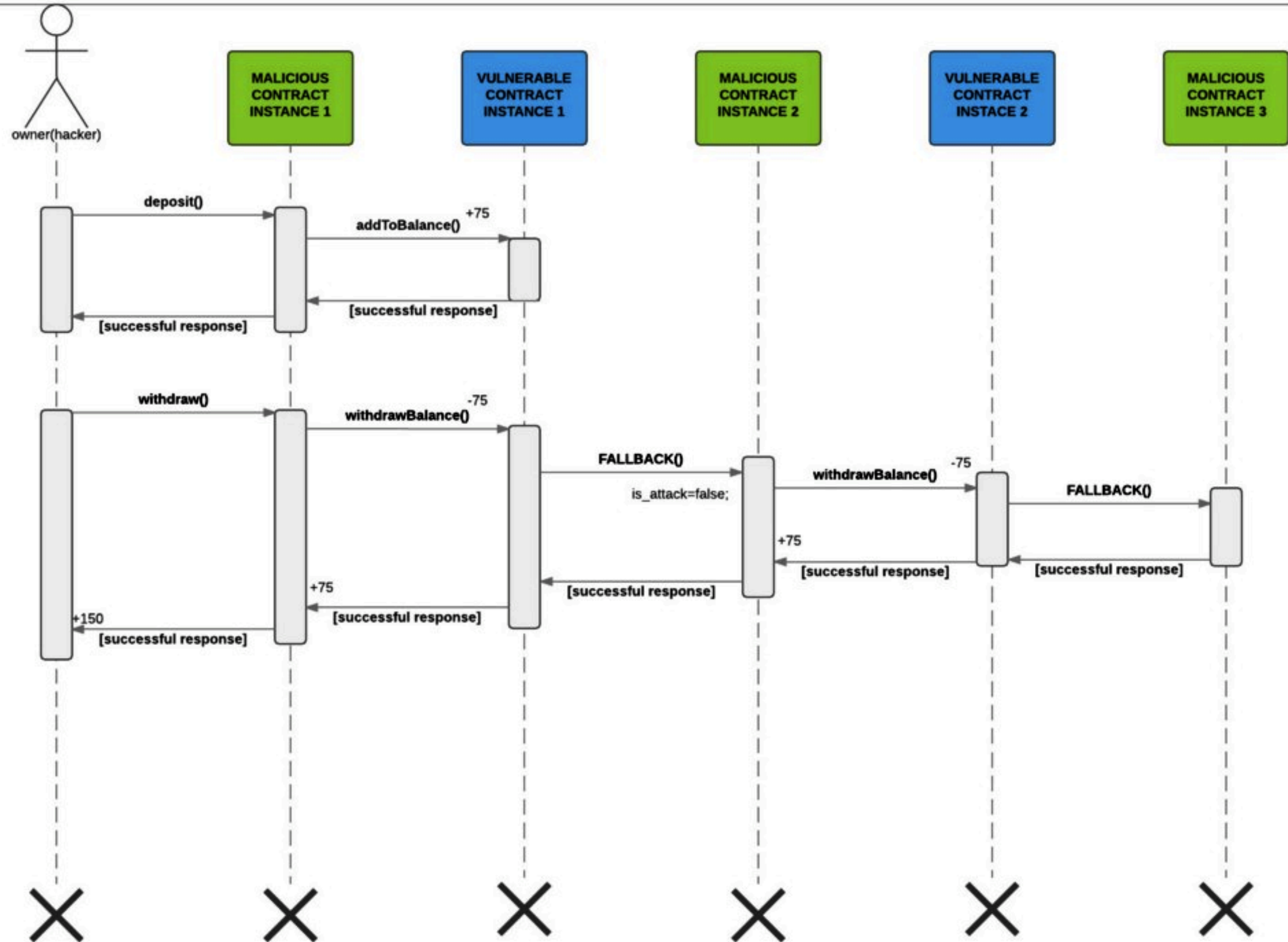
Violates the invariant

The DAO Bug

Attack Execution Trace



The DAO Bug



The DAO Bug

Object DAO

```
Map<Object, int> credit
```

```
int balance
```

```
Invariant (sum o: credit[o]) = balance
```

Method withdrawAll(Object o)

```
2: if (oCredit > 0)
```

```
// 2.5: credit[o] = 0
```

```
3: this.balance -= oCredit
```

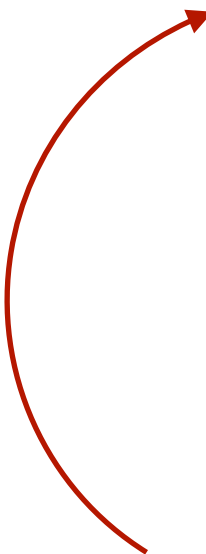
```
4: o.pay(oCredit)
```

```
5: credit[o] = 0
```

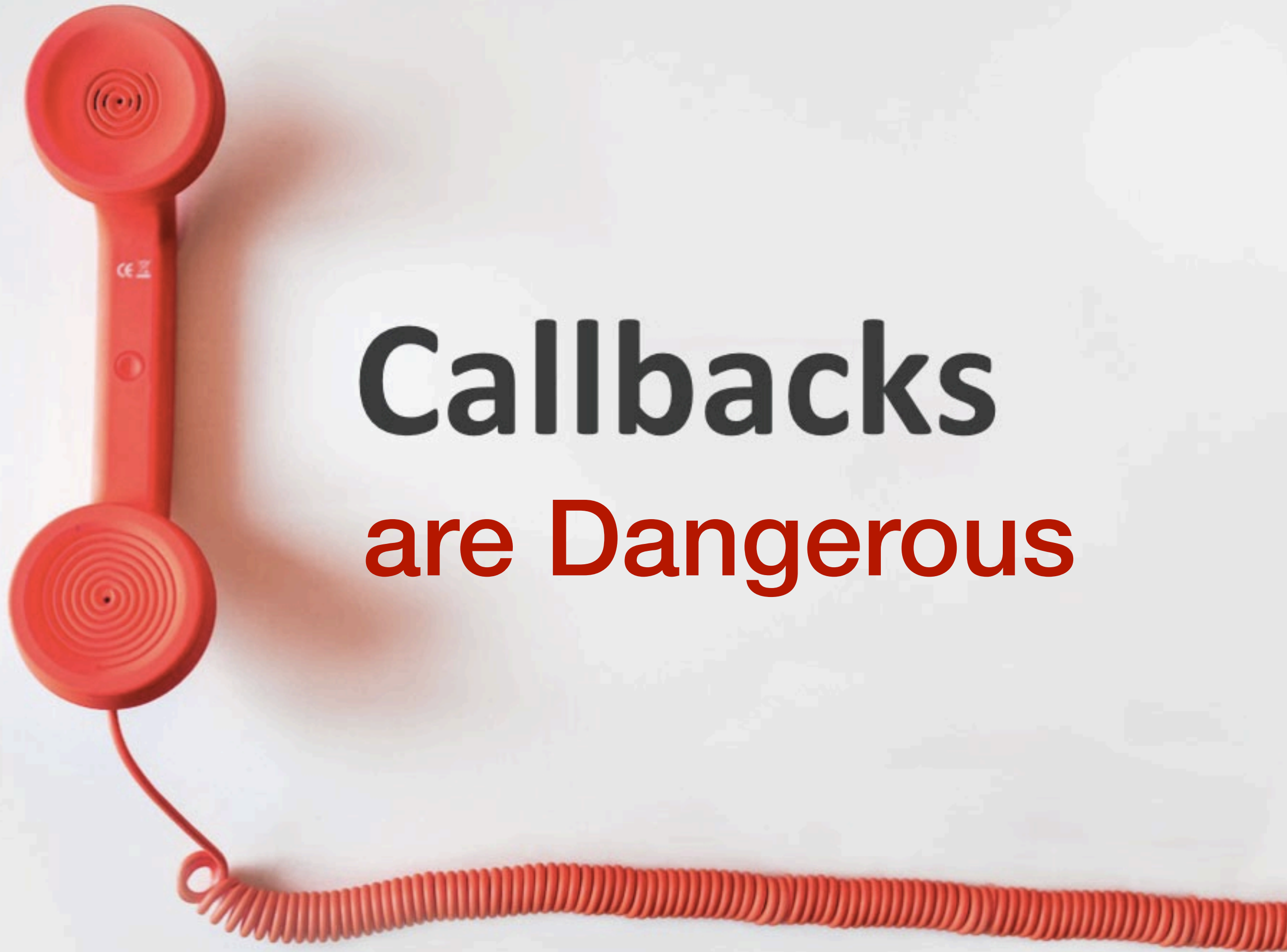
Method deposit(Object o, int amount)

```
6: credit[o] += amount
```

```
7: this.balance += amount
```



Adding this line fixes the bug



Callbacks
are Dangerous

ECF Contracts

- **ECF** - **E**ffectively **C**allback **F**ree
- **dECF** - **D**ynamically **E**ffective **C**allback **F**ree
A contract is dECF if there exists an equivalent execution of the contract without callbacks.
- **sECF** - **S**tatically **E**ffectively **C**allback **F**ree
A contract is sECF if all possible executions are ECF.

The DAO contract is not ECF

Dynamic Verification

- sECF for Ethereum contracts is not generally decidable - since the EVM is Turing Complete
- sECF is statically decidable for finite state space
 - Decidable for objects that can be modeled using Pushdown Automata (PDA)
- Instead, we implemented dynamic verification

Dynamic Verification

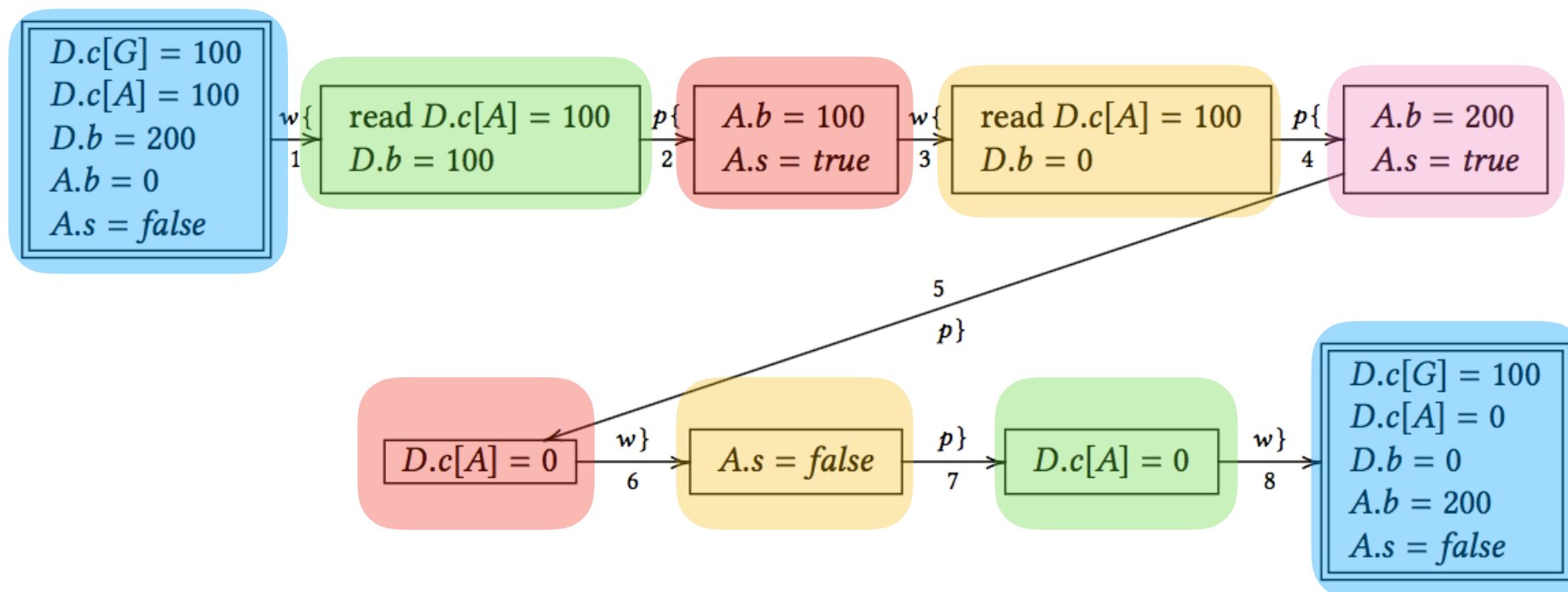
- *Segment*: represents the maximal sequence of adjacent transitions pertaining to the same object.
- A segment $t=(R,W,D,idx)$ contains info. about read $R(t)$ and write-sets $W(t)$, depth of invocation $D(t)$ and the segment execution index.
- *Commutative Segments*: Segments t_1 and t_2 commute, if

$$R(t_1) \cap W(t_2) = \emptyset \wedge R(t_2) \cap W(t_1) = \emptyset$$

Segments

```
Method withdrawAll(Object o)
  2: if (oCredit > 0)
    // 2.5: credit[o] = 0
  3: this.balance -= oCredit
  4: o.pay(oCredit)
  5: credit[o] = 0
```

```
Object Attacker
Object Dao, bool stop, int balance
Method init(Object dao)
  1: Dao = dao
  2: stop = false
Method pay(int profit)
  3: this.balance += profit
  4: if (!stop)
    5: stop = true
    6: Dao.withdrawAll(this)
  7: stop = false
```



Algorithm

- *Invocation Order Constraint (IOC)*: specifies invocation ordering as a graph
- Add *IOC*-s based on discovered commutativity of segments

Theorem. If the *IOC*-s graph contains no cycles, the execution is ECF.

Evaluation

- Prototype implementation of detection algorithm based on [geth Ethereum client](#)
- 2.2GHz Intel Xeon E5 x 2 (22 cores each, 2 threads / core), 256 GB RAM
- Ran on the entire blockchain from July 30, 2015 to March 30, 2017
- [CPU overhead: 3.5%. Memory overhead: 17%.](#)
- < 0.01% of executions were non-ECF
- [Only 10 out of 100M executions were legitimately non-ECF](#)

Blockchain	Date	Contracts	Executions	Callbacks	Non-ECF (%)
Ethereum	30.VII.2015-30.III.2017	138,457	81,097,421	128,670	3,315 (0.004%)
Ethereum	30.III.2017-23.VI.2017	203,859	15,311,650	155,662	6 (<0.001%)
Eth. Classic	30.VII.2015-29.VI.2017	91,191	32,494,464	81,731	2,288 (0.007%)

non-ECF Contracts

	Name	Contract address	Execs.	Execs. w. cbs.	Non-ECF	Stack depth
<i>Ethereum Network (ETH)</i>						
Vulnerable. Fixed.	C1	0xd654bdd32fc99471455e...	924	143	10	3
DAO	C2	0xbb9bc244d798123fde78...	274,820	103,064	3,296	2-146
DAO related	C3	0x34a5451ef61a567ee088...	91	8	1	46
Dark DAO	C4	0x304a554a310c7e546dfe...	13,223	2,812	1	3
DAO-like Exercise	C5	0x59752433dbe28f5aa59b...	15	6	1	3
Ambisafe	C6	0x97361ea911d6348cf2af...	44	42	6	2
Validity Labs training	C7	0xbf78025535c98f4c605f...	25	22	3	3-9
Ambisafe	C8	0x232f3a7723137ced12bc...	144	142	1	2
	C9	0x7c525c4e3b273a3afc4b...	35	33	2	2
<i>Ethereum Classic Network (ETC)</i>						
	C1	0xd654bdd32fc99471455e...	850	143	10	3
	C2	0xbb9bc244d798123fde78...	195,428	86,573	805	2-146
	C3	0x34a5451ef61a567ee088...	18	9	1	46
	C4	0x304a554a310c7e546dfe...	14,150	3,064	1	3
	C10	0xf4c64518ea10f995918a...	428	177	11	42-122
	C11	0xb136707642a4ea12fb4b...	2,582	305	201	17-20
	C12	0x0e0da70933f4c7849fc0...	5,330	3,992	1,259	12-57

non-ECF Contracts

```
Object C
  Object Sender
  Method call(data, sender)
    if (Sender != nil) throw
    Sender = sender; ret = this.do(data); Sender = nil

  Method do(data)
    ... // read Sender
```

**Valid Ambisafe contracts with measures to prevent DAO-like vulnerability.
This programming pattern is inherently non-ECF.**

Related Tools

- Oyente: Solidity verification tool
- Porosity: EVM bytecode decompiler

Conclusion

- In principal, automatic verification requires reasoning about the whole code
- We explored modularity, and..
- Obtained a general result that applies to objects in other programming languages

ANY
QUESTIONS
?